

```
In [1]: import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import random
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from collections import deque
```

```
In [2]: # =====
# 1. Carregar dataset
# =====
df = pd.read_csv("../creditcard.csv")

X = df.drop("Class", axis=1).values
y = df["Class"].values

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
In [3]: # =====
# 2. Definir ambiente
# =====
class FraudEnv:
    def __init__(self, X, y):
        self.X = X
        self.y = y
        self.n_samples = len(X)
        self.idx = 0

    def reset(self):
        self.idx = np.random.randint(0, self.n_samples)
        state = self.X[self.idx]
        label = self.y[self.idx]
        return state, label

    def step(self, action):
        label = self.y[self.idx]
        reward = 1 if action == label else -1
        done = True # cada amostra é um episódio
        next_state, next_label = self.reset()
        return next_state, reward, done, next_label
```

```
In [4]: # =====
# 3. Rede Q
# =====
class QNetwork(nn.Module):
    def __init__(self, input_dim, hidden_dim=64, output_dim=2):
        super(QNetwork, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
```

```

        nn.Linear(hidden_dim, hidden_dim),
        nn.ReLU(),
        nn.Linear(hidden_dim, output_dim) # Q(s, a) para 2 ações
    )
def forward(self, x):
    return self.fc(x)

```

```

In [5]: # =====
# 4. Deep Q-Learning
# =====
def train_dqn(X_train, y_train, episodes=50, gamma=0.99, epsilon=1.0,
              epsilon_min=0.01, epsilon_decay=0.995, batch_size=64):

    env = FraudEnv(X_train, y_train)
    state_dim = X_train.shape[1]
    action_dim = 2

    q_net = QNetwork(state_dim, 128, action_dim)
    target_net = QNetwork(state_dim, 128, action_dim)
    target_net.load_state_dict(q_net.state_dict())

    optimizer = optim.Adam(q_net.parameters(), lr=1e-3)
    memory = deque(maxlen=200) #maxLen=20000
    criterion = nn.MSELoss()

    all_rewards = []

    for ep in range(episodes):
        state, label = env.reset()
        state = torch.FloatTensor(state).unsqueeze(0)

        ep_reward = 0

        for t in range(1): # cada episódio = 1 transação
            # Exploração vs Exploração
            if random.random() < epsilon:
                action = np.random.randint(0, action_dim)
            else:
                with torch.no_grad():
                    q_vals = q_net(state)
                    action = torch.argmax(q_vals).item()

            next_state, reward, done, next_label = env.step(action)
            next_state = torch.FloatTensor(next_state).unsqueeze(0)

            memory.append((state, action, reward, next_state, done))
            ep_reward += reward

            state = next_state

        # Treinar se tiver memória suficiente
        if len(memory) > batch_size:
            batch = random.sample(memory, batch_size)
            states, actions, rewards, next_states, dones = zip(*batch)

            states = torch.cat(states)
            actions = torch.LongTensor(actions).unsqueeze(1)
            rewards = torch.FloatTensor(rewards).unsqueeze(1)
            next_states = torch.cat(next_states)
            dones = torch.BoolTensor(dones).unsqueeze(1)

```

```

        q_values = q_net(states).gather(1, actions)
        next_q_values = target_net(next_states).max(1)[0].unsqueeze(1)
        targets = rewards + gamma * next_q_values * (~done)

        loss = criterion(q_values, targets)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # Atualizar target network
    if ep % 50 == 0:
        target_net.load_state_dict(q_net.state_dict())

    # Decaimento do epsilon
    if epsilon > epsilon_min:
        epsilon *= epsilon_decay

    all_rewards.append(ep_reward)
    if (ep+1) % 500 == 0:
        print(f"Episode {ep+1}/{episodes}, Reward:
{np.mean(all_rewards[-500:]).3f}, Epsilon: {epsilon:.3f}")

    return q_net, all_rewards

```

```

In [6]: # =====
# 5. Treinar agente
# =====
q_net, rewards = train_dqn(X_train, y_train, episodes=300) #episodes=3000

# =====
# 6. Avaliar no teste
# =====
def evaluate(q_net, X, y):
    correct = 0
    total = 0
    q_net.eval()
    with torch.no_grad():
        for i in range(len(X)):
            state = torch.FloatTensor(X[i]).unsqueeze(0)
            q_vals = q_net(state)
            action = torch.argmax(q_vals).item()
            if action == y[i]:
                correct += 1
            total += 1
    return correct / total

acc = evaluate(q_net, X_test, y_test)
print(f"Accuracy no teste: {acc:.4f}")

```

Accuracy no teste: 0.9988

In []: